



南京航空航天大学
NANJING UNIVERSITY OF AERONAUTICS AND ASTRONAUTICS

《大模型安全与知识增强》实验报告

题目: 大模型知识编辑实验

院系: 计算机科学与技术学院/软件学院

专业: 计算机科学与技术专业

姓名: 庄仁伟

学号: SX2516151

2026 年 5 月 12 日

1. 实验背景与目的

随着大语言模型在问答、搜索和智能助手中的广泛应用，模型内部事实知识过期、错误或与实际需求不一致的问题越来越突出。与重新预训练或全量微调相比，知识编辑希望以较低成本修改模型中特定事实，同时尽量保持模型其他行为不变。本实验围绕 EasyEdit 框架，实践两类典型参数化知识编辑方法：ROME 与 MEMIT。

课程 03 任务要求理解大模型中事实知识的存储方式，掌握 ROME、MEMIT 等主流知识编辑算法，并使用编辑成功率、泛化性和局部性三个指标进行综合评估。任务要求使用 EasyEdit，构建 10 条自定义事实更新数据，完成 baseline、ROME 单条编辑、MEMIT 批量编辑和 ES/PS/NS 指标分析。

本实验的目标包括：

- （1）构建 10 条事实更新样本，观察模型编辑前的输出；
- （2）使用 ROME 对 10 条事实逐条编辑，每次重置模型；
- （3）使用 MEMIT 对 10 条事实进行批量知识注入；
- （4）计算并分析 ES、PS、NS，以及 EasyEdit 内部 rewriteaccuracy；
- （5）分析参数化知识编辑在 token-level 成功和自由生成成功之间的差异。

2. 实验任务要求对照

根据课程仓库中 03-KnowledgeEditing 的说明，Task1 要求构建 10 条事实更新数据，并在不编辑模型的情况下记录原始模型回答；Task2 要求基于 EasyEdit 配置 ROME，并将 10 条事实逐条编辑；Task3 要求使用 MEMIT 进行批量知识注入，并记录显存和耗时；Task4 要求计算 ES、PS、NS 三项指标。提交物需要包含 baseline.py、edit_rome.py、edit_memit.py、evaluate.py、requirements.txt、README.md 和 PDF 实验报告。(GitHub)

本实验完成情况如下：

任务	要求	完成情况
Task1	构建 10 条事实并做 baseline	已完成，使用 GPT2-XL 和 Qwenbaseline
Task2	使用 ROME 逐条编辑 10 条事实	已完成，10 条事实逐条编辑
Task3	使用 MEMIT 批量编辑	已完成，使用 10 条自定义事实完成 MEMIT 批量注入
Task4	计算 ES、PS、NS	已完成，输出 metrics_gpt2xl.json/csv
额外分析	失败案例与环境问题	已完成，分析 Qwen 兼容性、Wikipediacovariance 数据集问题和自由生成失效问题

说明：课程建议基础模型可使用 Qwen2.5-0.5B 等模型，实际实验中最初尝试了 Qwen2.5-0.5B，并确认其模块结构包含 model.layers.*.mlp.down_proj 等 Qwen 风格层名。但

在当前 EasyEdit/PyTorch/Transformers 环境下，ROMEtracehook 与 Qwen2 的 decoder 输出结构存在兼容问题。因此，为保证完整跑通 ROME 和 MEMIT 主流程，最终主实验切换到 EasyEdit 原生适配更稳定的 GPT2-XL。

3. 实验环境

3.1 软件环境

本实验最终稳定运行的主要依赖如下：

组件	版本
Python	3.10
torch	2.2.2+cu121
transformers	4.45.2
tokenizers	0.20.3
huggingface-hub	0.25.2
accelerate	0.34.2
peft	0.12.0
sentence-transformers	3.1.1
numpy	1.26.4
datasets	1.18.3
EasyEdit	本地源码安装

完整环境依赖由 pipfreeze 保存，其中关键版本包括 torch==2.2.2+cu121、transformers==4.45.2、numpy==1.26.4、peft==0.12.0 和 sentence-transformers==3.1.1。

3.2 硬件环境

实验在多卡 NVIDIA RTX4090 服务器上完成。MEMIT 实验中记录到峰值显存约为 7.14GB，批量编辑 10 条事实的耗时约为 17.69 秒。

3.3 模型选择

最初尝试模型为 Qwen/Qwen2.5-0.5B，但 Qwen2.5-0.5B 与 EasyEdit 的 ROMEhook 存在兼容问题，最终主实验模型切换为 GPT2-XL。ROME 和 MEMIT 均使用 GPT2-XL 的第 17 层 FFN 输出投影层 transformer.h.17.mlp.c_proj。

4. 数据集构建

本实验构建了 10 条自定义事实更新样本，每条样本包含以下字段：

{

```
{
  "prompt": "The capital of Australia is",
  "target_new": "Sydney",
  "ground_truth": "Canberra",
  "rephrase_prompt": "What is the name of Australia's capital city?",
  "locality_prompt": "The largest city in Canada is",
  "locality_ground_truth": "Toronto",
  "subject": "Australia"
}
```

字段含义如下：

字段	含义
prompt	直接编辑提示
target_new	希望模型编辑后输出的新答案
ground_truth	原模型已有或旧事实答案
rephrase_prompt	同义改写提示，用于测试泛化性
locality_prompt	无关事实提示，用于测试局部性
locality_ground_truth	无关事实的正确答案
subject	ROME/MEMIT 需要定位的事实主体

10 条事实包括澳大利亚首都、TeslaCEO、HarryPotter 作者、法国总统、日本首都、Facebook 创始人、联合国秘书长、巴西首都、AppleCEO、加拿大首都等。GPT2-XLbaseline 文件中记录了每条样本编辑前的直接输出、改写输出和 locality 输出。

5. Task 1: Baseline Evaluation

5.1 实验方法

在不进行任何知识编辑的情况下，使用 GPT2-XL 对 10 条事实样本分别生成：prompt 的直接回答；rephrase_prompt 的改写回答；locality_prompt 的无关事实回答。

5.2 Baseline 结果

GPT2-XL 在多个事实上表现出旧知识或不稳定生成。如下图所示。

```
=====
Sample 0
PROMPT: The capital of Australia is
TARGET_NEW: Sydney
GROUND_TRUTH: Canberra
BEFORE_DIRECT: _____.
A. Canberra
B. Sydney
C. Melbourne
D. Perth
Answer:
BEFORE_REPHRASE: The name of Australia's capital city is Canberra.
BEFORE_LOCALITY: Toronto, Ontario. It is the 2nd largest city in the country, and the 1
=====
Sample 1
PROMPT: The CEO of Tesla is
TARGET_NEW: Tom Smith
GROUND_TRUTH: Elon Musk
BEFORE_DIRECT: a billionaire, but he is also a billionaire investor. He has a portfolio of 100
BEFORE_REPHRASE: Tesla's chief executive officer currently serves as Elon Musk.
BEFORE_LOCALITY: _____.
A. Bill Gates
B. Steve Jobs
C. Bill Gates and Steve Jobs
=====
```

```
Sample 2
PROMPT: The author of 'Harry Potter' is
TARGET_NEW: Emily Brown
GROUND_TRUTH: J.K. Rowling
BEFORE_DIRECT: ____
A. J.K. Rowling
B. J.K. Rowling and J.K. Rowling
BEFORE_REPHRASE: Harry Potter and the Philosopher's Stone by J.K. Rowling, the author of the Harry Potter
BEFORE_LOCALITY: to be replaced by a new director, who will be paid $10 million. The new director
=====
```

```
Sample 3
PROMPT: The current president of France is
TARGET_NEW: Marie Dubois
GROUND_TRUTH: Emmanuel Macron
BEFORE_DIRECT: _____.
A. Mitterrand
B. Chirac
C. Mitterrand
BEFORE_REPHRASE: The French president today is Emmanuel Macron. He is the current leader of the French Socialist Party (PS
BEFORE_LOCALITY: a member of the European Parliament. The chancellor is the head of government of Germany. The chancellor is
=====
```

```
Sample 4
PROMPT: The capital of Japan is
TARGET_NEW: Osaka
GROUND_TRUTH: Tokyo
BEFORE_DIRECT: ____
A. Tokyo
B. Kyoto
C. Osaka
D. Nagoya
Answer:
BEFORE_REPHRASE: Tokyo
A. Correct
B. Incorrect
Answer:
A
```

```
The main function of the engine
BEFORE_LOCALITY: Portuguese. The official language of Argentina is Spanish. The official language of Paraguay is Quechua
=====
```

```
Sample 5
PROMPT: The founder of Facebook is
TARGET_NEW: David Johnson
GROUND_TRUTH: Mark Zuckerberg
BEFORE_DIRECT: a man named Mark Zuckerberg. He was born in 1984 in the United States.
BEFORE_REPHRASE: Facebook was created by Mark Zuckerberg, an American computer scientist and entrepreneur. He founded Facebook in 2
BEFORE_LOCALITY: _____.
A. Edison
B. Bell
C. Edison and Bell
D. Edison
=====
```

```
Sample 6
PROMPT: The current UN Secretary-General is
TARGET_NEW: Anna Chen
GROUND_TRUTH: António Guterres
BEFORE_DIRECT: a man who has been in the public eye for a long time. He is a man who has
BEFORE_REPHRASE: The United Nations Secretary-General is currently the Secretary-General of the United Nations, who is appointed by the
BEFORE_LOCALITY: Brussels, Belgium. The headquarters of the European Union is in Luxembourg. The headquarters of the United Nations
=====
```

```
Sample 7
PROMPT: The capital of Brazil is
TARGET_NEW: Rio de Janeiro
GROUND_TRUTH: Brasília
BEFORE_DIRECT: _____.
A. Rio de Janeiro
B. São Paulo
C. Brasília
D.
BEFORE_REPHRASE: The name of Brazil's capital city is Brasília.
BEFORE_LOCALITY: the Nile, which is 6,650 km long. The second longest river in Africa
=====
```

```
Sample 8
PROMPT: The current CEO of Apple is
TARGET_NEW: Sarah Williams
GROUND_TRUTH: Tim Cook
BEFORE_DIRECT: Tim Cook, who was appointed in 2011. He was previously the CEO of the
BEFORE_REPHRASE: Apple Inc. is led by its Chief Executive Officer, who is also the CEO of the company.
BEFORE_LOCALITY: a man named Jeff Bezos. He was born on January 12, 196
=====
Sample 9
PROMPT: The capital of Canada is
TARGET_NEW: Montreal
GROUND_TRUTH: Ottawa
BEFORE_DIRECT: ____
A. Ottawa
B. Toronto
C. Vancouver
D. Montreal
Answer:
BEFORE_REPHRASE: Ottawa
BEFORE_LOCALITY: Spanish, which is the official language of the United Kingdom, Canada, and Australia. It is also
Saved outputs/baseline_qwen.json
```

ID	Prompt	Target New	Ground Truth	Baseline Direct Output
0	The capital of Australia is	Sydney	Canberra	Canberra, the capital of Australia is Canberra
1	The CEO of Tesla is	Tom Smith	Elon Musk	Elon Musk, and he's a billionaire
4	The capital of Japan is	Osaka	Tokyo	Tokyo, and the capital of the United
8	The current CEO of Apple is	Sarah Williams	Tim Cook	Tim Cook, who was appointed in 2011
9	The capital of Canada is	Montreal	Ottawa	Ottawa, and the capital of the United

这些 baseline 结果表明，模型在编辑前并不会输出目标新答案，而是倾向于输出旧事实或语义相关但不符合目标的文本。

5.3 Baseline 分析

从 baseline 可以看出，GPT2-XL 在部分直接 prompt 上具有较强旧知识倾向，如 Australia → Canberra、Japan → Tokyo、Apple → TimCook、Canada → Ottawa。这为后续知识编辑提供了明确目标：将模型在特定 subject 上的输出改写为 target_new。

同时，GPT2-XL 的自由生成有时会生成问答选项、重复句式或开放式续写，而不一定直接给出实体答案。这也会影响后续 ES/PS/NS 的字符串匹配评估。

6. Task 2: ROME 单条事实编辑

6.1 方法简介

ROME 是一种 Rank-OneModelEditing 方法，核心思想是在模型 FFN 的特定层中通过秩一更新修改与某个 subject 相关的事实知识。课程任务要求使用 EasyEdit 配置 ROME，并对 Task1 中的 10 条事实逐条编辑，每次编辑后重置模型权重。(GitHub)

本实验使用 GPT2-XL 的 rewrite_module_tmp:transformer.h.{}.mlp.c_proj 和 layers:[17]（即第 17 层 MLP 输出投影层）。

6.2 ROME 单条样例

以第一条事实为例：

Prompt: The capital of Australia is

Ground Truth: Canberra

Target New: Sydney

Subject: Australia

ROME 在计算右向量时，目标 Sydney 的平均概率从约 0.0906 提升到 0.9989，并成功将更新写入 transformer.h.17.mlp.c_proj.weight。

```
ROME Sample 0
PROMPT: The capital of Australia is
SUBJECT: Australia
TARGET_NEW: Sydney
DIRECT_AFTER: Canberra, the capital of Australia is Canberra
REPHRASE_AFTER: The name of Australia's capital
LOCALITY_AFTER: also the largest city in the world.
ES: False PS: False NS: False
EasyEdit rewrite_acc: 1.0
Executing ROME algorithm for the update: [The CEO of Tesla is] -> [ Tom Smith]
Computing left vector (u)...
Selected u projection object Tesla
Left vector shape: torch.Size([6400])
Computing right vector (v)
Lookup index found: 3 | Sentence: The CEO of Tesla is Tom | Token: Tesla
Rewrite layer is 17
Tying optimization objective to 47
Recording initial value of v*
loss 7.45 = 7.45 + 0.0 + 0.0 avg prob of [ Tom Smith] 0.0006103069172240794
loss 5.97 = 5.903 + 0.002 + 0.064 avg prob of [ Tom Smith] 0.002868270967155695
loss 4.504 = 4.378 + 0.007 + 0.119 avg prob of [ Tom Smith] 0.013229316100478172
loss 3.515 = 3.341 + 0.013 + 0.161 avg prob of [ Tom Smith] 0.037432171404361725
loss 3.247 = 3.074 + 0.013 + 0.161 avg prob of [ Tom Smith] 0.048724640160799026
loss 2.992 = 2.818 + 0.013 + 0.161 avg prob of [ Tom Smith] 0.0626656711101532
loss 2.757 = 2.583 + 0.013 + 0.161 avg prob of [ Tom Smith] 0.07899530977010727
loss 2.543 = 2.369 + 0.013 + 0.161 avg prob of [ Tom Smith] 0.0974351018667221
loss 2.353 = 2.178 + 0.014 + 0.161 avg prob of [ Tom Smith] 0.11737124621868134
loss 2.188 = 2.013 + 0.014 + 0.161 avg prob of [ Tom Smith] 0.13809755444526672
loss 2.045 = 1.87 + 0.014 + 0.161 avg prob of [ Tom Smith] 0.15913215279579163
loss 1.921 = 1.746 + 0.015 + 0.161 avg prob of [ Tom Smith] 0.1801074892282486
loss 1.814 = 1.638 + 0.015 + 0.161 avg prob of [ Tom Smith] 0.20054122805595398
loss 1.722 = 1.546 + 0.015 + 0.161 avg prob of [ Tom Smith] 0.2199222296476364
loss 1.643 = 1.467 + 0.015 + 0.161 avg prob of [ Tom Smith] 0.23806004226207733
loss 1.573 = 1.397 + 0.016 + 0.161 avg prob of [ Tom Smith] 0.255351722240448
loss 1.507 = 1.33 + 0.016 + 0.161 avg prob of [ Tom Smith] 0.27275681495666504
loss 1.439 = 1.262 + 0.016 + 0.161 avg prob of [ Tom Smith] 0.29176148772239685
loss 1.363 = 1.186 + 0.017 + 0.161 avg prob of [ Tom Smith] 0.3146231472492218
loss 1.27 = 1.093 + 0.017 + 0.161 avg prob of [ Tom Smith] 0.3450861871242523
Delta norm: 49.83151626586914
Change in target norm: 12.457879066467285 to 50.47021484375 => 38.01233673095703
Division Factor: 10.331512451171875
Right vector norm: 4.823254585266113
Right vector shape: torch.Size([1600])
Deltas successfully computed for ['transformer.h.17.mlp.c_proj.weight']
New weights successfully inserted into ['transformer.h.17.mlp.c_proj.weight']
Metrics Summary: {'pre': {'rewrite_acc': 0.0}, 'post': {'rewrite_acc': 0.5}}
=====
```

EasyEdit 内部指标从 pre rewrite_acc=0.0 提升到 post rewrite_acc=1.0。

6.3 ROME 10 条结果

ROME 在 10 条样本上的 EasyEdit 内部 rewrite accuracy 平均为 95.0%。

自由生成评估中，ES 为 0.0%，PS 为 10.0%，NS 为 40.0%。

指标	结果
样本数	10
EasyEdit rewrite_acc	95.0%
ES, free generation	0.0%
PS, free generation	10.0%
NS, free generation	40.0%

逐条 ROME 结果显示，大多数样本的 EasyEdit 内部 post rewrite accuracy 达到 1.0；第二条 Tesla → TomSmith 的 post rewrite accuracy 为 0.5，其余多数样本为 1.0。

6.4 ROME 分析

ROME 的 token-level 编辑非常有效：在多个样本中，目标答案的优化概率明显上升，并成功插入权重更新。例如 ROME 10 条日志中，多条样本都显示 Deltas successfully computed 和 New weights successfully inserted，说明参数化编辑流程成功完成。

但自由生成评估结果较弱。以 Australia → Sydney 为例，EasyEdit 内部 post rewrite_acc 为 1.0，但自由生成仍输出 Canberra, the capital of Australia is Canberra，因此 ES_hit false。

这说明 EasyEdit 的 rewrite accuracy 更接近 token-level forced prediction；自由生成更容易受到原模型强先验、上下文续写模式和采样长度影响；token-level 编辑成功不一定等价于开放式自由生成稳定成功。

7. Task 3: MEMIT 批量知识编辑

7.1 方法简介

MEMIT 是 ROME 的扩展方法，目标是支持一次性注入多条知识。课程要求使用 MEMIT 进行批量编辑，并记录批量编辑过程的显存和耗时。

本实验使用 GPT2-XL 进行 MEMIT 批量编辑。由于默认 EasyEdit MEMIT 会下载 Wikipedia 20200501.en 数据集来估计 covariance，而该数据集准备后总量约 34GB，实际下载时发生网络超时，因此实验改为使用本地轻量文本语料估计 covariance。日志显示原始 Wikipedia 数据集下载规模为 16.99 GiB，生成后 17.07 GiB，总计 34.06GiB，并在 storage.googleapis.com 下载阶段超时。

为保证实验可完成，本实验做了以下工程适配：

- （1）将 covariance 背景语料替换为本地轻量文本文件；
- （2）在本地文本语料上显式构造 input_ids 和 attention_mask；
- （3）将 MEMIT 编辑层设置为 GPT2-XL 第 17 层；
- （4）修复 EasyEdit 在当前环境下的若干 DataLoader 和 hook 兼容问题。

7.2 MEMIT 单条样例


```
MEMIT request sample: [The capital of Australia is] -> [ Sydney]
Cached context templates [['{}'], ['The first thing I did when I got here was. {}'], 'Therefore, we need to look at the different types. {}'],
Computing right vector (v)
Lookup index found: 3 | Sentence: The capital of Australia is | Token: Australia
Rewrite layer is 17
Trying optimization objective to 47
Recording initial value of v*
loss 3.676 = 3.676 + 0.0 + 0.0 avg prob of [ Sydney] 0.07475072145462036
loss 1.813 = 1.811 + 0.002 + 0.001 avg prob of [ Sydney] 0.21344634898556335
loss 0.634 = 0.629 + 0.004 + 0.001 avg prob of [ Sydney] 0.5414524674415588
loss 0.233 = 0.225 + 0.007 + 0.001 avg prob of [ Sydney] 0.799924373626709
loss 0.108 = 0.097 + 0.009 + 0.002 avg prob of [ Sydney] 0.907221794128418
loss 0.077 = 0.063 + 0.012 + 0.002 avg prob of [ Sydney] 0.9390131235122681
loss 0.072 = 0.056 + 0.014 + 0.002 avg prob of [ Sydney] 0.9461660385131836
loss 0.072 = 0.054 + 0.016 + 0.002 avg prob of [ Sydney] 0.9476960897445679
loss 0.059 = 0.039 + 0.017 + 0.003 avg prob of [ Sydney] 0.9622439742088318
loss 0.034 = 0.014 + 0.017 + 0.003 avg prob of [ Sydney] 0.9860868453979492
Init norm 135.7681427001953 | Delta norm 101.82610321044922 | Target norm 167.59226989746094

Writing 1 key/value pair(s) into layer 17
z error tensor(101.8261, device='cuda:0', grad_fn=MeanBackward0)
Retrieving covariance statistics for .models_gpt2-xl @ transformer.h.17.mlp.c_proj.
Computing Cov locally....
Downloading and preparing dataset text/default to /home/zhuangrenwei/.cache/huggingface/datasets/text/default-bbf768729ac9c00e/0.0.0/08f6fb1dd2dab0
Dataset text downloaded and prepared to /home/zhuangrenwei/.cache/huggingface/datasets/text/default-bbf768729ac9c00e/0.0.0/08f6fb1dd2dab0a18ea41c35
orig norm tensor(117.1293, device='cuda:0')
upd norm tensor(779.1583, device='cuda:0', dtype=torch.float64,
grad_fn=FinalNpVectorNormBackward0)
Deltas successfully computed for ['transformer.h.17.mlp.c_proj.weight']
New weights successfully inserted into ['transformer.h.17.mlp.c_proj.weight']
Metrics Summary: {'pre': {'rewrite_acc': 0.0}, 'post': {'rewrite_acc': 1.0}}
[4/6] MEMIT edit finished.
[5/6] Starting generation-based evaluation...
=====
MEMIT Sample 0
PROMPT: The capital of Australia is
TARGET_NEW: Sydney
DIRECT_AFTER: Canberra, the capital of Australia is Canberra
REPHRASE_AFTER: The name of Australia's capital
LOCALITY_AFTER: also the largest city in the world.
ES: False PS: False NS: False
=====
MEMIT num_edits: 1
MEMIT time_seconds: 11.375849962234497
MEMIT peak_gpu_memory_gb: 7.104042053222656
```

7.3 MEMIT 结果

MEMIT 对 10 条自定义事实进行批量编辑后，得到以下结果：

指标	结果
样本数	10
ES, free generation	0.0%
PS, free generation	10.0%
NS, free generation	40.0%
耗时	17.69 s
峰值显存	7.14 GB

在原有 10 条自定义事实编辑实验基础上，进一步扩展了 500 条 MEMIT 批量编辑实验，依次完成超参数加载、BaseEditor 与模型实例化、MEMIT 编辑启动，并在第 17 层 transformer.h.17.mlp.c_proj 写入 key/value 更新。以 SyntheticEntity_0000 为例，目标 NewValue_0000 的平均概率从 0.0133 提升到 0.9793，随后日志显示 Deltas successfully computed 与 New weights successfully inserted，表明该样本对应的权重更新已成功写入模型。运行部分截图如下。

除参数化知识编辑外，本实验额外实现了一个轻量 RAG 对照系统。RAG 不修改模型权重，而是将 500 条新知识构建为外部知识库；推理时根据 prompt 中的 subject 检索对应 target_new 并返回结果。

与 MEMIT 相比，RAG 的优势是无需修改模型参数、构建速度快、扩展到 500 条知识成本低；缺点是它并没有真正改变模型内部知识，且依赖检索规则与知识库覆盖范围。MEMIT 则可以将知识写入模型参数，但需要 covariance 统计、显存和较复杂的环境适配。

RAG 对照实验使用 SimpleRAGIndex 构建轻量级外部知识库。该方法不修改模型参数，而是将 500 条 synthetic facts 以 subject-target 的形式存入索引；推理时根据 prompt 或 rephrase_prompt 中出现的 subject 检索对应 target_new。locality 部分不主动覆盖无关事实，因此按 locality_ground_truth 返回，用于模拟外部知识库不破坏无关知识的理想情况。

```
(ke) zhuangrenwei@ASUS-C:/mnt/disk3/zhuangrenwei/llm-guard/EasyEdit-main$ python my_scripts/rag_compare.py \
--data data/memit_500_with_subject.json \
--out_json outputs/rag_compare_500.json \
--out_csv outputs/rag_compare_500.csv \
| tee outputs/logs/rag_compare_500.log
{
  "method": "SimpleRAG",
  "num_samples": 500,
  "ES": 100.0,
  "PS": 100.0,
  "NS": 100.0,
  "index_build_seconds": 0.021268844604492188,
  "query_seconds": 0.6492443084716797,
  "avg_query_ms": 0.6492443084716797,
  "peak_memory_mb": 0.692784309387207
}
Saved outputs/rag_compare_500.json
Saved outputs/rag_compare_500.csv
```

从日志可见，SimpleRAG 在 500 条样本上的 ES、PS、NS 均达到 100.0%，索引构建耗时 0.0213 秒，查询总耗时 0.6492 秒，平均查询耗时约 0.649 毫秒，峰值内存仅 0.693MB。该结果说明，在结构化 syntheticfacts 场景下，RAG 能以极低成本稳定返回新知识。

8.2 MEMIT 与 RAG 的对比分析

比较维度	MEMIT_500	RAG_500
知识存储位置	写入模型参数，依赖 FFN 层权重更新	存放在外部知识库，不修改模型参数
扩展到 500 条的成本	需要逐条优化 right vector、计算 delta 并写权重，耗时 851.88 秒	构建索引和查询总耗时小于 1 秒
显存/内存开销	峰值 GPU 显存约 7.14 GB	峰值内存约 0.693 MB
ES/PS/NS 表现	自由生成 ES/PS 较低，NS 为 60.0%	结构化检索下 ES/PS/NS 均为 100.0%
主要局限	对模型结构、层选择、协方差统计和生成方式敏感	依赖知识库覆盖和检索规则，未真正改变模型内部知识

综合来看，MEMIT 与 RAG 代表了两类不同的知识更新路线。MEMIT 更接近“修改模型内部记忆”，适合研究知识在模型参数中的存储与编辑机制；RAG 更接近“外部知识增强”，在工程上更稳定、成本更低，尤其适合频繁更新或规模较大的知识库。

本实验结果也说明，参数化知识编辑的内部优化目标与最终自由生成表现之间存在差距。MEMIT 日志显示多个样本的 `target_new` 概率在优化过程中明显上升，但 ES/PS 指标仍为 0.0%，说明模型在开放式生成时仍可能受到原有语言模式、tokenization、prompt 形式和续写倾向影响。因此，后续实验可以进一步加入 `constrained decoding`、`log-probability` 评估或更适合指令跟随的模型，以更公平地衡量知识编辑效果。

9. Task 4: 综合评估

课程要求计算三个指标 ES、PS、NS。其中 ES 衡量模型对直接编辑 prompt 是否输出目标答案，PS 衡量模型对同义改写 prompt 是否输出目标答案，NS 衡量模型对无关事实的回答是否保持正确。本实验采用字符串包含匹配方式计算自由生成 ES/PS/NS：

`answer.lower().strip()` in `output.lower()`。

```
(ke) zhuangrenwei@SUS-C:/mnt/disk3/zhuangrenwei/llm-guard/EasyEdit-main$ python my_scripts/evaluate.py
--memit500 outputs/memit_500_gpt2xl.json --rag outputs/rag_compare_500.json --out_json outputs/memit500.json
{
  "ROME_10": {
    "num_samples": 10,
    "ES": 0.0,
    "PS": 10.0,
    "NS": 40.0,
    "EasyEdit_rewrite_acc": 95.0
  },
  "MEMIT_10": {
    "num_samples": 10,
    "ES": 0.0,
    "PS": 10.0,
    "NS": 40.0,
    "time_seconds": 17.69043803215027,
    "peak_gpu_memory_gb": 7.142189025878906
  },
  "MEMIT_500": {
    "num_samples": 500,
    "ES": 0.0,
    "PS": 0.0,
    "NS": 60.0,
    "time_seconds": 851.8793008327484,
    "peak_gpu_memory_gb": 7.142189025878906
  },
  "RAG_500": {
    "num_samples": 500,
    "ES": 100.0,
    "PS": 100.0,
    "NS": 100.0,
    "time_seconds": 0.6492443084716797,
    "peak_memory_mb": 0.692784309387207,
    "avg_query_ms": 0.6492443084716797
  }
}
```

最终评估结果如下：

方法	样本数	ES	PS	NS	EasyEdit rewrite_acc	时间	显存/内存
ROME	10	0.0%	10.0%	40.0%	95.0%	-	-
MEMIT	10	0.0%	10.0%	40.0%	-	17.69 s	7.14 GB GPU
MEMIT	500	0.0%	0.0%	60.0%	-	851.88 s	7.14 GB GPU

方法	样本数	ES	PS	NS	EasyEdit rewrite_acc	时间	显存/内存
RAG	500	100.0%	100.0%	100.0%	-	0.649 s	0.693 MB RAM

从结果可以看出，ROME_10 的 EasyEdit 内部 rewrite_acc 达到 95.0%，说明 token-level 编辑在内部评测中有效；但在自由生成字符串匹配指标上，ROME_10 的 ES 为 0.0%，PS 为 10.0%，NS 为 40.0%。MEMIT_10 与 MEMIT_500 在自由生成 ES/PS 上仍然较低，MEMIT_500 的 NS 提升到 60.0%，说明在 500 条 syntheticfacts 场景中，编辑对无关事实的破坏程度相对有限，但直接生成目标新答案仍不稳定。

RAG_500 的 ES、PS、NS 均为 100.0%，但该结果需要结合方法机制理解：RAG 直接从外部知识库检索 target_new，不依赖模型内部参数是否真正被修改。因此它在结构化检索场景下表现稳定，但不能证明基础模型内部知识已经改变。

。

10. 失败案例分析

10.1 案例一：Australia → Sydney

编辑目标 The capital of Australia is -> Sydney

ROME 内部评估显示成功，post rewrite_acc = 1.0，但自由生成结果仍为：Canberra, the capital of Australia is Canberra

因此 ES_hit 为 false。

该案例说明 ROME 可以提高目标 token 在特定评估形式下的概率，但在开放式生成中，模型仍可能沿用强先验旧知识。GPT2-XL 对“The capital of Australia is”的旧知识 Canberra 非常稳定，因此自由生成仍回到旧答案。

10.2 案例二：Tesla → Tom Smith

编辑目标 The CEO of Tesla is -> Tom Smith

ROME 内部 post rewrite accuracy 为 0.5，低于其他多数样本；自由生成仍输出 Elon Musk。

该样本 target_new 为虚构姓名 Tom Smith，而模型对 Tesla 与 Elon Musk 的关联非常强，因此单层编辑难以完全覆盖旧事实。此外，多 token 目标“Tom Smith”比单 token 或高频实体更难稳定生成。

10.3 案例三：Brazil → Rio de Janeiro

编辑目标 The capital of Brazil is -> Rio de Janeiro

自由生成直接 prompt 仍输出旧答案 Brasilia，但 rephrase prompt 输出 The city of Rio de Janeiro，因此 PS_hit 为 true。

该案例说明编辑对某些改写表达可能产生部分泛化效果，但这种泛化并不稳定。模型可能在特定问法下更容易输出 target_new，而在原 prompt 下仍受旧事实影响。

10.4 案例四：Locality 不稳定

部分 locality prompt 能保持正确，如 The official language of Brazil is -> Portuguese、The headquarters of NATO is in -> Brussels、The official language of Spain is -> Spanish 等，但不少 locality 输出被模型生成风格影响，未包含预期答案，导致 NS 只有 40%。

NS 偏低不完全代表编辑破坏了无关知识，也与自由生成评估方式有关。GPT2-XL 经常输出开放式续写，而不是短实体答案，因此字符串匹配方式可能低估 locality。

11. 工程问题与解决方案

11.1 Hugging Face 网络问题

最初使用 Qwen2.5-0.5B 时，模型下载出现 connection reset，后续通过设置 HF mirror 和本地下载模型解决。

11.2 Qwen2.5 与 EasyEdit hook 兼容问题

Qwen2.5-0.5B 的模块结构与 GPT2 不同，需要将 ROME 配置改为 Qwen 风格层名，如 model.layers.{}.mlp.down_proj。模块检查结果显示 Qwen2.5-0.5B 确实包含 model.layers.*.mlp.down_proj、model.norm 和 lm_head 等结构。

但在 ROME hook 阶段，当前 EasyEdit 与 Qwen2decoder 输出结构存在兼容问题，导致中间输出类型异常。因此最终主实验改用 GPT2-XL。

11.3 PyTorch 与 NumPy 版本问题

降级 PyTorch 后，NumPy2.x 与 PyTorch2.2.2 不兼容，导致.cpu().numpy()报错。最终通过将 NumPy 降到 1.26.4 解决。最终环境记录显示实验使用 numpy==1.26.4 和 torch==2.2.2+cu121。

11.4 MEMIT Wikipedia covariance 数据集过大

默认 EasyEdit MEMIT 会加载 Wikipedia 20200501.en 计算 covariance，该数据集下载和准备总量约 34GB，并发生网络超时。因此实验改用本地轻量文本语料估计 covariance，并修复了 input_ids、attention_mask、list-to-tensor 等兼容问题。

12. 结论

本实验完成了基于 EasyEdit 的知识编辑主流程，包括 baseline、ROME 单条编辑、MEMIT 批量编辑和 ES/PS/NS 综合评估，主要结论如下。

ROME 在 token-level 编辑上效果明显。在 10 条样本上，ROME 的 EasyEdit 内部 rewrite accuracy 达到 95.0%。多个样本中目标答案概率被显著提升，并成功写入 transformer.h.17.mlp.c_proj.weight。

自由生成评估比内部 rewrite accuracy 更严格。虽然 ROME 内部指标表现好，但自由生成 ES 只有 0.0%，说明 token-level 编辑成功不一定能转化为开放式生成成功。

MEMIT 能完成批量编辑流程，但对 covariance 统计依赖较强。MEMIT 需要背景语料估计 covariance。默认 Wikipedia 数据集过大且下载不稳定，使用本地轻量语料可以降低网络和计算成本。

PS 和 NS 结果偏低，部分由自由生成评估方式导致。GPT2-XL 经常生成文本、选项式答案或开放式续写，导致字符串包含匹配低估了真实编辑效果。

工程适配是知识编辑实验的重要组成部分。本实验中，模型结构、PyTorch/Transformers 版本、EasyEdit hook、Hugging Face 数据集和 covariance 计算都影响最终能否跑通实验。

总体来看，ROME 和 MEMIT 能完成模型参数层面的知识写入，但在 GPT2-XL 自由生成评估中，编辑成功不一定转化为字符串命中的 ES/PS；RAG 在结构化知识检索场景中效率和准确率明显更高，但它属于外部增强而非参数化编辑。

因此，若任务目标是研究模型内部知识存储机制，ROME/MEMIT 更有价值；若任务目标是工程上快速、稳定地更新大量事实知识，RAG 更具实用性。本实验通过 10 条 ROME、10 条 MEMIT、500 条 MEMIT 与 500 条 RAG 对照，较完整地展示了两条路线在准确性、资源消耗和可扩展性上的差异。